

Протокол до лабораторної роботи 4

Тема: Мікросервіси з використанням Messaging Queue

Дата: 19 квітня 2026

Гілка: `micro_mq`

Коміт реалізації: `d26b545`

Статус: реалізацію завершено, основні сценарії перевірено

GitHub репозиторій: <https://github.com/senivan/spa-lab-4.git>

1. Мета роботи

Розширити попередню мікросервісну систему так, щоб:

- запис транзакцій у `counter-service` відбувався асинхронно через чергу повідомлень;
- `facade-service` для читання даних і далі використовував HTTP GET;
- адреси мікросервісів не були захардкожені у `facade-service`, а отримувались через `config-server`;
- система зберігала працездатність при тимчасовій недоступності `counter-service`.

2. Що було реалізовано

2.1. Новий `config-server`

Додано окремий сервіс `config-server`, який виконує роль простого реєстру сервісів.

Реалізовані endpoint-и:

- `POST /register`
Реєстрація сервісу у форматі: `json { "service_name": "logging-service", "service_url": "http://logging1:8001" }`
- `GET /services/{service_name}`
Повернення списку всіх зареєстрованих інстансів сервісу.

2.2. Самореєстрація мікросервісів

При старті сервісів:

- `facade-service`
- `counter-service`
- `logging-service` (усі 3 екземпляри)

вони автоматично виконують `POST /register` до `config-server`.

2.3. Service discovery у facade-service

Раніше facade-service використовував захардкожені адреси logging-service і counter-service.

Тепер логіка така:

- перед зверненням до logging-service або counter-service виконується запит до config-server;
- для logging-service із поверненого списку випадково вибирається один інстанс;
- для counter-service також використовується адреса з реєстру.

2.4. Messaging Queue між facade-service і counter-service

У якості MQ використано Hazelcast Queue.

Зміни:

- POST /transaction у facade-service більше не викликає counter-service напямую;
- після логування транзакції facade-service поміщає повідомлення в чергу counter-tx-queue;
- counter-service у фоновому режимі читає повідомлення з черги та застосовує їх до PostgreSQL.

Схема роботи:

1. клієнт надсилає POST /transaction до facade-service;
2. facade-service вибирає один з logging-service через config-server і зберігає транзакцію в Hazelcast Map;
3. facade-service ставить транзакцію в Hazelcast Queue;
4. counter-service читає транзакцію з черги;
5. counter-service оновлює баланс у PostgreSQL.

2.5. Поведінка GET /user/{user_id}

Для читання даних логіка залишилась синхронною:

- список транзакцій читається з logging-service;
- баланс читається з counter-service.

Якщо counter-service тимчасово недоступний, facade-service повертає:

```
{
  "balance": null,
  "transactions": [...],
  "counter_available": false
}
```

Це відповідає вимозі лабораторної роботи повертати `null` або еквівалентну ознаку недоступності.

3. Архітектура системи

```
Клієнт
|
v
facade-service (8000)
| \
|  \--> config-server (8500) -> отримання адрес сервісів
|
+--> logging-service[1..3] -> збереження транзакцій у Hazelcast Map
|
+--> Hazelcast Queue -> асинхронна передача транзакцій
                        |
                        v
                    counter-service (8002)
                        |
                        v
                    PostgreSQL (5432)

Hazelcast cluster:
- hz1:5701
- hz2:5701
- hz3:5701
```

4. Склад docker-compose

Піднімаються такі сервіси:

- config-server
- facade-service
- counter-service
- logging1
- logging2
- logging3
- hz1
- hz2
- hz3
- postgres

Host ports:

- facade-service -> 8000
- counter-service -> 8002
- logging1 -> 8001
- logging2 -> 8003

- logging3 -> 8004
- config-server -> 8500

Примітка: у вихідному compose з попередньої роботи був конфлікт host-порту 8002 між logging2 і counter-service. Для коректного запуску його виправлено.

5. Основні файли, які були змінені

- [config-server/app/main.py](#)
- [facade-service/app/main.py](#)
- [counter-service/app/main.py](#)
- [logging-service/app/main.py](#)
- [docker-compose.yml](#)

6. API та приклади запитів

6.1. Запис транзакції

```
curl -X POST http://localhost:8000/transaction \
-H "Content-Type: application/json" \
-d '{
  "user_id": "user1",
  "amount": 5,
  "message": "msg1"
}'
```

Приклад відповіді:

```
{
  "transaction_id": "1776585715815158091",
  "queued": true,
  "logging": {
    "ok": true,
    "instance": "logging3",
    "transaction_id": "1776585715815158091"
  }
}
```

Пояснення:

- `queued: true` означає, що транзакція успішно поставлена в MQ;
- поле `logging.instance` показує, який екземпляр `logging-service` обробив запит.

6.2. Читання балансу та історії транзакцій

```
curl http://localhost:8000/user/user1
```

Приклад відповіді:

```
{
  "balance": 12,
  "transactions": [
    {
      "transaction_id": "1776585715794361590",
      "timestamp": "2026-04-19T08:01:55",
      "user_id": "user1",
      "amount": 7,
      "message": "msg2"
    },
    {
      "transaction_id": "1776585715815158091",
      "timestamp": "2026-04-19T08:01:55",
      "user_id": "user1",
      "amount": 5,
      "message": "msg1"
    }
  ],
  "counter_available": true
}
```

6.3. Читання при недоступному counter-service

Після `docker pause` для `counter-service`:

```
curl http://localhost:8000/user/user3
```

Отримана відповідь:

```
{
  "balance": null,
  "transactions": [
    {
      "transaction_id": "1776585737943127587",
      "timestamp": "2026-04-19T08:02:17",
      "user_id": "user3",
      "amount": 10,
      "message": "msg-paused"
    }
  ],
  "counter_available": false,
  "counter_error": ""
}
```

Після `docker unpause` для `counter-service`:

```
curl http://localhost:8000/user/user3
```

Отримана відповідь:

```
{
  "balance": 10,
  "transactions": [
    {
      "transaction_id": "1776585737943127587",
      "timestamp": "2026-04-19T08:02:17",
      "user_id": "user3",
      "amount": 10,
      "message": "msg-paused"
    }
  ],
  "counter_available": true
}
```

7. Фрагменти логів

7.1. Реєстрація сервісів у config-server

```
[config-server] registered logging-service -> http://logging1:8001
[config-server] registered logging-service -> http://logging2:8001
[config-server] registered logging-service -> http://logging3:8001
[config-server] registered facade-service -> http://facade-service:8000
[config-server] registered counter-service -> http://counter-service:8002
```

7.2. Використання різних екземплярів logging-service

```
[facade-service] logging via http://logging1:8001
[facade-service] logging via http://logging3:8001
```

Це підтверджує, що facade-service не працює з одним жорстко заданим інстансом, а вибирає сервіс з реєстру.

7.3. Поміщення транзакцій у чергу

```
[facade-service] queued transaction 1776585725867718804 for counter-service
[facade-service] queued transaction 1776585737943127587 for counter-service
```

7.4. Обробка транзакцій counter-service

```
[counter-service] user2 += 1 -> 1
[counter-service] consumed queued transaction 1776585725867718804
[counter-service] user2 += 1 -> 2
[counter-service] consumed queued transaction 1776585725937368054
[counter-service] user3 += 10 -> 10
[counter-service] consumed queued transaction 1776585737943127587
```

8. Перевірка вимог завдання

Вимога 1. Розгорнути Messaging Queue

Виконано.

Використано `Hazelcast Queue` з назвою `counter-tx-queue`.

Вимога 2. `POST` у `facade-service` додає повідомлення до черги

Виконано.

`facade-service` після логування транзакції викликає `put()` у `Hazelcast Queue`.

Вимога 3. `counter-service` читає повідомлення як `consumer`

Виконано.

Запущено фоновий цикл читання черги з подальшим оновленням PostgreSQL.

Вимога 4. `GET` залишився HTTP-запитом до `counter-service`

Виконано.

Читання балансу відбувається через HTTP `GET /balance/{user_id}`.

Вимога 5. Додати `config-server`

Виконано.

Усі сервіси реєструються через `POST /register`, а `facade-service` читає адреси через `GET /services/{service_name}`.

Вимога 6. Показати, що різні `logging-service` отримують повідомлення

Виконано.

У логах зафіксовано використання щонайменше `logging1` і `logging3`.

Вимога 7. Перевірка відмовостійкості

Виконано.

При `docker pause` для `counter-service`:

- `POST /transaction` продовжує повертати успіх;
- `GET /user/{user_id}` повертає `balance: null`;
- після `docker unpause` накопичена транзакція обробляється;
- баланс стає коректним.

9. Команди для запуску

Запуск системи

```
docker compose up -d --build
```

Перевірка стану контейнерів

```
docker compose ps
```

Перегляд логів

```
docker compose logs -f
```

Симуляція відмови counter-service

```
docker pause 04-msg-q-counter-service-1
```

Повернення сервісу в роботу

```
docker unpause 04-msg-q-counter-service-1
```

10. Висновок

У лабораторній роботі реалізовано асинхронну взаємодію між facade-service і counter-service через Hazelcast Queue, а також додано config-server для service discovery.

Система тепер підтримує:

- динамічний вибір інстансів logging-service;
- асинхронний запис транзакцій;
- накопичення повідомлень при тимчасовій недоступності counter-service;
- коректне відновлення після повернення сервісу в роботу.

Поставлені у завданні вимоги реалізовані.